ANTEATER CHESS
SOFTWARE SPECIFICATIONS
VERSION 1.1

TEAM 21

Naveen Khan
Mia Ness
Austin Nguyen
Queency Tan
Aman Macwan
Gurvin Dhanjal

TABLE OF CONTENTS

1. SOFTWARE ARCHITECTURE OVERVIEW

1.1 Main Data Types and Structures - 3
1.2 Major Software Components - 4
1.3 Module Interfaces - 5
1.4 Overall Program Control Flow - 7

2. INSTALLATION

2.1 System Requirements and Compatibility - 8
2.2 Setup and Configuration - 8
2.3 Building, Compilation, and Installation - 9

3. DOCUMENTATION OF PACKAGES, MODULES, AND INTERFACES

3.1 Detailed Description of Data Structures - 10
3.2 Detailed Description of Functions and Parameters - 13
3.3 Detailed Description of Input and Output Formats - 16

4. DEVELOPMENT PLAN AND TIMELINE

4.1 Partitioning of Tasks - 17
4.2 Team Member Responsibilities - 19

5. BACK MATTER

5.1 Copyright - 20
6. References - 21
7. Index - 22

Software Architecture Overview

1.1 MAIN DATA TYPES AND STRUCTURES

The system is built around a small set of data structures that together capture the complete state of an in-progress Anteater Chess game. These structures are designed to keep the chess rules, the GUI, the AI, and the move history loosely coupled while still sharing a single, consistent representation of the game.

The core of the system is a pair of enumerated types, Color and Rank. Color identifies which player owns a piece (WHITE or BLACK). Rank identifies the kind of piece (KING, QUEEN, ANTEATER, BISHOP, KNIGHT, ROOK, PAWN), with EMPTY reserved for unoccupied squares. A third enumeration, Piece_Type, combines color and rank into a single value used as an index into the GUI's piece-image array.

Each piece in the game is represented by a Piece struct. The struct stores the piece's color and rank, a pointer to its Cairo image used by the renderer, an integer point value, the piece's current Pos on the board, a moved flag (used to validate castling), and a pointer to a PieceVTable. The PieceVTable holds two function pointers — canMove and canCapture — that implement the piece-specific movement and capture rules. This indirection lets every piece be handled uniformly while still following its own rules.

The board itself is a Board struct containing a two-dimensional 8x10 array of Piece pointers; an empty square is represented by a NULL entry. A Board_State struct tracks the dynamic state that is not part of the board itself: the currently clicked piece, the list of legal destination squares for that piece, the total number of moves made, the most recent Move (for en passant detection), and flags such as gameOver and hasUpdate that drive the GUI.

A Board_Bundle struct ties everything together for the GUI. It owns pointers to the Board, the Board_State, every GtkWidget that needs to be updated (board drawing area, move log text view, promotion popover, per-side timer labels), the array of piece images, and the user-selectable game settings (user color, CPU color, CPU difficulty, and a humanVsHuman flag). Almost every callback in the program receives a Board_Bundle pointer.

Moves are represented by a Move struct containing the start and end coordinates plus boolean flags for capture, enPassant, and castle. A separate Undo_Record struct stores everything required to reverse a single move — the Move itself, pointers to the moved and captured pieces, and additional fields (movedRook, rookMove, movedPieceMovedBefore, rookMovedBefore) needed to correctly undo castling. A LIFO stack of Undo_Records implements the undo button.

1.2 Major Software Components

The program is organized as a set of modules, each responsible for a single aspect of the chess game. The split exists to reduce coupling and to make each module replaceable without affecting the others.

The major components are:

- **GUI Application (gui/gui.c) —** Holds main(), creates the GTK 4 application, builds the main window with the board, move log, timers, and undo / quit buttons, displays the Game Setup and Game Over dialogs, and routes user input to the rest of the system.
- **Game Controller (game_controller.c) —** Translates a single click on the board into a complete game action. Decides whether the click selects a piece, executes a move, triggers a special move, or starts a chain of Anteater captures.
- **Board Management (board.c) —** Owns the Board struct. Provides board initialization, square lookup and mutation, piece movement and capture utilities, and pawn promotion.
- **Board State (board_state.c) —** Owns the Board_State struct. Tracks the currently clicked piece, generates the list of legal moves for highlighting, and exposes update / game-over flags consumed by the GUI.
- **Board Bundle (board_bundle.c) —** Container struct that holds everything else (board, state, widgets, images, timer data, game settings). Centralizes initialization and provides getter / setter helpers.
- **Piece Logic (pieces.c) —** Defines the Piece struct, the Pos struct, all per-piece canMove / canCapture rules, en passant and castling rule checks, and the king-in-check helpers (findKing, attackSquare, kingCheck).
- **Move Processing (move.c) —** Defines the Move struct. Executes validated moves, performs ordinary captures, sideways pawn captures, en passant, castling, and Anteater chain captures. Provides possibleMove for stalemate / checkmate detection.
- **Renderer (gui/render.c) —** Cairo drawing routines for the board, squares, pieces, and legal-move highlights. Loads piece textures from gui/resources/.
- **Color Theming (gui/color_gui.c) —** Centralizes the board color scheme and CSS class names used by the GUI.
- **AI / Computer Player (ai.c) —** Generates a move for a given color at a given AIDifficulty (Easy: random; Medium: greedy heuristic; Hard: alpha-beta minimax to depth 2).
- **Logging (log.c) —** Maintains the human-readable chess_game.log file. Provides newLog plus addMove, addCapture, addCastle, logEnPassant, logPromotion, logCheck, logCheckmate, logDraw, and logQuit.
- **Undo (undo.c) —** Stack-backed history of executed moves. Supports push / pop and applies the inverse of any single Undo_Record, including castles and Anteater chain captures.
- **Save / Load (save.c) —** Planned. Will write the entire Board, current turn, and move log to a text file under save/ and reload it to resume play.

The module hierarchy is:

- GUI Application (gui/gui.c)
    - Game Controller (game_controller.c)
        - Move Processing (move.c)
            - Piece Logic (pieces.c)

        - Board Management (board.c)
        - Logging (log.c)
        - Undo (undo.c)
    - Board State (board_state.c)
        - Piece Logic (pieces.c)
    - AI / Computer Player (ai.c)
        - Piece Logic (pieces.c)
        - Move Processing (move.c)
    - Renderer (gui/render.c)
    - Color Theming (gui/color_gui.c)

1.3 Module Interfaces

Selected public function prototypes for each module. Helpers and getters / setters are omitted; full declarations live in the corresponding header files.

GUI Application (gui/gui.c)

```
int  main(int argc, char **argv);
static void activate(GtkApplication *app, gpointer user_data);
void onClick(GtkGestureClick *g, int n, double x, double y, gpointer ud);
gboolean onTimerTick(gpointer user_data);
void appendToLogUI(Board_Bundle *boardData);
void createPopUp(Board_Bundle *boardData);
```

Game Controller (game_controller.c)

```
void sendInput(Board_Bundle *boardData, Pos clickPos);
```

Board Management (board.c)

```
void   defaultInitializeBoard(Board *board);
void   initializeBoard(Board *board, Icon **images);
Piece *getSquare(const Board *board, int row, int col);
void   setSquare(Board *board, Piece *piece, int row, int col);
bool   hasPiece(const Board *board, int row, int col);
Piece *deletePiece(Board *board, Pos pos);
void   addPiece(Board *board, Piece *piece, Pos pos);
void   movePiece(Board *board, Board_State *state, Piece *p, Pos newPos);
void   capturePiece(Board *board, Board_State *state, Piece *p, Pos at);
void   promotePiece(Board_Bundle *bundle, Pos pos, Rank newRank);
```

Board State (board_state.c)

```
void initializeBoardState(Board_State *state);
void generateLegalMoves(Board_State *state, Board *b, Piece *p,
                        Pos start, Move *lastMove);
bool isLegalMoveSquare(const Board_State *state, Pos pos);
bool isPromotion(const Board_State *state);
void incrementMovesMade(Board_State *state);
```

Piece Logic (pieces.c)

```
void posCtor(Pos *p, int row, int col);
void pieceCtor(Piece *p, Icon *img, Color c, Rank r, Pos pos);
PieceVTable *getVtable(Rank type);
int  rookCanMove   (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  bishopCanMove (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  knightCanMove (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  queenCanMove  (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  kingCanMove   (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  pawnCanMove   (Board *b, Piece *p, int sr,int sc,int er,int ec);
int  anteaterCanMove(Board *b, Piece *p, int sr,int sc,int er,int ec);
int  pawnCanEnPassant(Board *b, Piece *p, int sr,int sc,int er,int ec,
                      Move *lastMove);
int  kingCanCastle (Board *b, Piece *p, int sr,int sc,int er,int ec);
Pos  findKing(Board *b, Color color);
int  kingCheck(Board *b, Color kingColor);
```

Move Processing (move.c)

```
void executeMove          (Board *b, Move *m, Move lastMove);
void executeCapture       (Board *b, Move *m);
void executePawnCapture   (Board *b, Move *m);
void executeAnteaterCapture(Board *b, Move *m);
void executeEnPassant     (Board *b, Move *m);
void executeCastle        (Board *b, Move *m);
int  anteaterHasAdjacentPawn(Board *b, Piece *anteater);
int  possibleMove(Board_State *st, Board *b, Color turn);
```

AI / Computer Player (ai.c)

```
int aiSelectMove(Piece board[8][10], Color player, Move *out);
int aiSelectMoveAtDifficulty(Piece board[8][10], Color player,
                             AIDifficulty difficulty, Move *out);
```

Logging (log.c)

```
const char newLog       (const char *filename);
const char addMove      (const char *f, int n, Piece *p, Pos to);
const char addCapture   (const char *f, int n, Piece *p, Pos from, Pos at);
const char addCastle    (const char *f, int n, Piece *p, int sideNum);
const char logEnPassant (const char *f, int n, Piece *p, Pos pos);
const char logPromotion (const char *f, int n, Piece *p, Pos to,
                          Piece *newPiece);
const char logCheck     (const char *f, int n, Piece *p);
const char logCheckmate (const char *f, int n, Piece *winner);
const char logDraw      (const char *f, int n);
const char logQuit      (const char *f, int n, Piece *quitter);
```

Undo (undo.c)

```
void undoSet(void);
void pushMoveForUndo(Board *board, Move *move);
int  undoPush(const Undo_Record *record);
int  undoPop (Undo_Record *out);
int  undo    (Undo_Record *rec, Board *board);
```

Renderer (gui/render.c)

```
void createImages(cairo_surface_t **imgs, int n);
int  pixToIndex(int pix);
int  indexToPix(int i);
void drawBoard(GtkDrawingArea *area, cairo_t *cr,
               int width, int height, gpointer data);
void drawSquare(GtkDrawingArea *area, cairo_t *cr, int row, int col);
void drawPiece (cairo_t *cr, Icon *img, int row, int col);
```

1.4 Overall Program Control Flow

1. Program start: gui/gui.c main() creates a GtkApplication, initializes a Board, a Board_State, and a Board_Bundle, loads piece textures into the Board_Bundle, and connects the activate signal.

2. activate() builds the main window (board drawing area, move log text view, white/black timer labels, undo button, quit button), wires up click and button callbacks, and presents the window.

3. A Game Setup dialog appears. The player selects:

- Game mode: Human vs CPU or Human vs Human
- If CPU mode: human color (White / Black) and CPU difficulty (Easy / Medium / Hard)

4. On Start Game, the dialog stores the chosen settings in the Board_Bundle, starts the 1-second timer tick (g_timeout_add_seconds), and (if the user chose Black in CPU mode) immediately triggers a CPU move.

5. Game loop: the program waits for user clicks. Each click on the board fires onClick, which:

- Ignores the click if the game is already over, a promotion popover is open, or it is the CPU's turn in CPU mode.
- Otherwise calls sendInput(boardData, clickPos) in game_controller.c.

6. sendInput dispatches on the click:

- If the click is on a friendly piece: select it, call generateLegalMoves, and request a redraw.
- If the click is on a legal destination: build a Move, call executeMove (which performs ordinary moves, captures, en passant, castling, and Anteater chain captures), update lastMove, and append a log entry via the appropriate log.c function.
- If the moved piece is an Anteater and another enemy Ant is orthogonally adjacent, the same piece is reselected and the chain continues; otherwise the turn count is incremented.
- If the moved piece is an Ant on the back rank, the promotion popover is opened and the user picks a piece type.

7. After a successful move, gui.c resets the moving side's timer, calls finishGameIfNeeded, and (if the game is not over) schedules the CPU's reply via cpuMoveCallback when in CPU mode.

8. finishGameIfNeeded inspects the position for the side to move next: if the King is missing or in checkmate, it shows the Game Over dialog with a winner message; if there are no legal moves and the King is not in check, it shows a stalemate message; if the King is in check, it logs a check announcement.

9. Timer tick: every second, onTimerTick decrements the active side's seconds; if a timer hits zero it shows the Game Over dialog declaring the opposing side the winner on time.

10. Game over: the dialog offers New Game (re-initialize the board, re-open the Game Setup dialog) or Quit (terminate the application). Quitting via the quit button on the board opens a confirmation dialog and, on Yes, calls g_application_quit.

2.1 System Requirements and Compatibility

Before installing and running the program please make sure your system meets the following requirements:

- Linux operating system (tested on UCI EECS lab machines)
- GCC compiler with C99 support
- GNU Make
- GTK 4 development libraries and pkg-config
- Cairo and GDK-Pixbuf (typically installed with GTK 4)
- At least a few megabytes of free disk space

2.2 Setup and Configuration

Step 1: Extract the source archive.

```
$ gtar -xvzf Chess_V1.0_src.tar.gz
```

Step 2: Change into the program directory.

```
$ cd Chess_V1.0_src
```

Step 3: Compile the program. The default target builds the GTK 4 GUI executable bin/chess; bin/chess_ascii is also available as a debug fallback that does not require GTK.

```
$ make           # builds bin/chess (GTK 4 GUI)
$ make gui       # explicitly builds bin/chess_gui
$ make ascii     # text-only debug build (bin/chess_ascii)
```

Step 4: (Optional) Run the test suite. test_rulecheck verifies per-piece movement and capture rules; test_boarddisplay verifies board rendering on a known layout.

```
$ make test
```

Step 5: Run the program.

```
$ ./bin/chess
```

2.3 Building, Compilation, and Installation

Once the source archive has been extracted and the working directory is the project root, the included Makefile drives the entire build. Three executables can be produced:

- bin/chess — default target. The full GTK 4 graphical client, built from gui/gui.c plus all of the board, piece, move, AI, log, undo, board_state, board_bundle, and game_controller modules.
- bin/chess_gui — alias for the GUI build, useful when both targets are present.
- bin/chess_ascii — text-only debug binary built from src/main.c. Skips board.c and the entire GUI; useful for unit-style testing without a display server.

Object files are compiled in place and linked directly into the executable; there is no separate install step. To remove all build artifacts:

```
$ make clean
```

To produce a redistributable source tarball:

```
$ make tar
```

3.1 Description of Data Structures

ENUMERATED TYPES

Three enumerations describe the core categorical state of the game.

```
typedef enum {
    WHITE = 0,
    BLACK = 1
} Color;

typedef enum {
    EMPTY    = -1,
    KING     = 0,
    QUEEN    = 1,
    ANTEATER = 2,
    BISHOP   = 3,
    KNIGHT   = 4,
    ROOK     = 5,
    PAWN     = 6
} Rank;

typedef enum {
    WHITE_KING = 0,    BLACK_KING = 1,
    WHITE_QUEEN = 2,   BLACK_QUEEN = 3,
    WHITE_ANTEATER = 4, BLACK_ANTEATER = 5,
    WHITE_BISHOP = 6,  BLACK_BISHOP = 7,
    WHITE_KNIGHT = 8,  BLACK_KNIGHT = 9,
    WHITE_ROOK = 10,   BLACK_ROOK = 11,
    WHITE_ANT = 12,    BLACK_ANT = 13
} Piece_Type;
```

Piece_Type is used to index into the GUI's piece-image array and into the PIECE_RESOURCES path table in render.c.

AI difficulty is a separate enumeration consumed only by ai.c:

```
typedef enum {
    AI_EASY,
    AI_MEDIUM,
    AI_HARD
} AIDifficulty;
```

VIRTUAL TABLES

Per-piece movement and capture rules are dispatched through a virtual table named PieceVTable. Each Piece points to the table that matches its rank, so move and capture validation can be invoked uniformly without a switch on Rank.

```
typedef struct PieceVTable {
    int (*canMove)   (Board *b, Piece *p, int sr,int sc,int er,int ec);
    int (*canCapture)(Board *b, Piece *p, int sr,int sc,int er,int ec);
} PieceVTable;
```

PIECE STRUCTURE

A single Piece carries everything the rules engine, the renderer, and the AI need to know about it:

```
struct Piece {
    Icon       *img;     // Cairo image used by the renderer
    Color       color;   // WHITE or BLACK
    Rank        type;    // KING, QUEEN, ANTEATER, ...
    PieceVTable *vtable; // canMove + canCapture for this Rank
    int         value;   // material value (used by the AI)
    Pos         pos;     // current square
    int         moved;   // 0 if never moved (used for castling)
};
```

POSITION STRUCTURE

Board coordinates are integer (row, col) pairs in the closed range row in [0,7] and col in [0,9]. row 0 is the black back rank and col 0 is file A.

```
typedef struct Pos {
    int row;
    int col;
} Pos;
```

MOVE STRUCTURE

A Move is the integer description of a single ply, including flags for the special moves the rules engine cares about.

```
typedef struct Move {
    int startRow;
    int startCol;
    int endRow;
    int endCol;
    int capture;    // 1 if the move captures
    int enPassant;  // 1 if the move is an en passant capture
    int castle;     // 1 if the move is a castle
} Move;
```

BOARD REPRESENTATION

The board is a struct wrapping a two-dimensional array of Piece pointers. NULL marks an empty square.

```
typedef struct Board {
    Piece *board[BOARD_HEIGHT][BOARD_WIDTH]; // 8 rows, 10 columns
} Board;
```

BOARD_HEIGHT and BOARD_WIDTH are defined in enum.h as 8 and 10 respectively.

BOARD STATE

Board_State holds the per-turn dynamic data that is not stored in the board array itself.

```
typedef struct Board_State {
    Piece *clickedPiece;
    bool   hasUpdate;
    bool   moveSuccess;
    bool   gameOver;
    Pos    clickPos;
    Pos    legalMoves[BOARD_HEIGHT * BOARD_WIDTH];
    int    legalMoveCount;
    int    movesMade;
    Move   lastMove;
} Board_State;
```

BOARD BUNDLE

Board_Bundle is the central object passed to virtually every GUI callback. It owns no widgets directly but stores pointers to all of the long-lived state the GUI needs to coordinate.

```
typedef struct Board_Bundle {
    Board       *board;
    Board_State *boardState;
    GtkWidget   *boardWidget;
    GtkTextView *log;
    GtkTextBuffer *logBuffer;
    GtkWidget   *promotionPopUp;
    Icon       **images;
    char        *move;          // last status string for the log
    GtkLabel    *whiteTimerLabel;
    GtkLabel    *blackTimerLabel;
    guint        timerSourceId;
    int          whiteSeconds;
    int          blackSeconds;
    Color        userColor;
    Color        cpuColor;
    bool         userStarts;
    bool         humanVsHuman;
    AIDifficulty cpuDifficulty;
} Board_Bundle;
```

UNDO RECORD AND STACK

Undo is implemented as a LIFO stack of Undo_Record values. A single record stores everything required to reverse one move, including the second piece and second move that participate in a castle.

```
typedef struct Undo_Record {
    Move    move;
    Piece  *movedPiece;
    Piece  *capturedPiece;
    Piece  *movedRook;            // castle: the rook involved
    Move    rookMove;             // castle: where the rook went
    int     movedPieceMovedBefore;// pre-move 'moved' flags
    int     rookMovedBefore;
} Undo_Record;
```

3.2 Detailed Description of Functions and Parameters

Function 1: Move

```
void executeMove(Board *board, Move *move, Move lastMove);
```

- **Description:** Executes a Move that has already been validated as legal.
- **Parameters:** the board, the move to execute, and the most recent move (used to recognize en passant).
- **Outputs:** the moved piece is placed on its destination square, captures (including en passant), castling, and Anteater chain captures are dispatched to their specialized executors, and the move is appended to chess_game.log.

Function 2: Capture

```
void executeCapture(Board *board, Move *move);
```

- **Description:** Performs an ordinary capture: removes the piece on the destination square and moves the attacker onto it.
- **Outputs:** the captured piece is freed and removed from the board, and a (capture) entry is appended to the log.

Function 3: Anteater Capture

```
void executeAnteaterCapture(Board *board, Move *move);int
anteaterHasAdjacentPawn(Board *board, Piece *anteater);
```

- **Description:** Captures a single Ant during an Anteater chain capture. After execution, anteaterHasAdjacentPawn determines whether another Ant is orthogonally adjacent to the Anteater; if so, the controller keeps the turn and lets the user click the next Ant.
- **Outputs:** the captured Ant is removed; if a chain is possible, "Anteater can capture again" is appended to the log.

Function 4: Castle

```
void executeCastle(Board *board, Move *move);int  kingCanCastle(Board *b, Piece *p,
int sr,int sc,int er,int ec);
```

- **Description:** Moves the King two squares toward a chosen Rook and places that Rook on the square the King crossed.
- **Logic:** kingCanCastle verifies that neither piece has moved before, the squares between them are empty, and the King is not in, through, or out of check. Only legal castles appear as highlighted destinations.

Function 5: En Passant

```
void executeEnPassant(Board *board, Move *move);int  pawnCanEnPassant(Board *b, Piece *p, int sr,int sc,int er,int ec, Move *lastMove);
```

- **Description:** Captures an enemy Ant that has just advanced two squares from its starting row.
- **Logic:** pawnCanEnPassant returns true only when lastMove is exactly such an advance. The capture is otherwise impossible on subsequent turns.

Function 6: Promotion

```
void promotePiece(Board_Bundle *bundle, Pos pos, Rank newRank);
```

- **Description:** Replaces the Ant at pos with a new piece of the chosen Rank in the same color.
- **Logic:** Triggered when executeMove leaves an Ant on the back rank. The GUI opens a promotion popover anchored to the destination square; the user's selection is forwarded to promotePiece.

Function 7: Check / Checkmate / Stalemate

```
int  kingCheck(Board *board, Color kingColor);int  possibleMove(Board_State *state, Board *board, Color turn);static bool finishGameIfNeeded(Board_Bundle *bundle, Color playerWhoMoved);
```

- **Description:** After every move, finishGameIfNeeded calls kingCheck on the side to move next. If that side has no legal moves (possibleMove returns 0), the game ends — checkmate if it was in check, otherwise stalemate.
- **Outputs:** the appropriate message is appended to the log and a Game Over dialog is presented.

Function 8: Per-Side Timer

```
gboolean onTimerTick(gpointer user_data);void     updateTimerLabels(Board_Bundle *bundle);
```

- **Description:** A 1-Hz timer source decrements the active side's remaining seconds. updateTimerLabels formats both labels as MM:SS and applies the timer-active CSS class to whichever side is to move.
- **Logic:** When a side's seconds reach zero the opposing side wins on time and the Game Over dialog is shown.

Function 9: Turn

```
static Color colorToMove(int movesMade);  // helper
```

- **Description:** Identifies whose turn it is. White moves on even movesMade values; Black on odd.
- **Logic:** Click events on the board are ignored unless the clicked piece's color matches colorToMove. In Human vs CPU mode, the board also ignores clicks whenever it is the CPU's turn.

Function 10: New Game

```
static void onEndGameDialogResponse(GtkDialog *d, int id, gpointer ud);static void
showStartupDialog(GtkWindow *parent, Board_Bundle *bundle);
```

- **Description:** Re-initializes the board, the board state, the log buffer, and both timers, then re-opens the Game Setup dialog.
- **Logic:** Triggered at program launch and from the New Game button on the Game Over dialog. If the user chooses Black in CPU mode, the CPU plays the first move automatically once the dialog closes.

Function 11: End Game

```
static void showEndGameDialog(Board_Bundle *bundle, const char *resultMsg);
```

- **Description:** Builds and presents a modal dialog containing the result message and the total number of moves played, with New Game and Quit buttons.

Function 12: Undo

```
void undoSet(void);void pushMoveForUndo(Board *board, Move *move);int
undoPop(Undo_Record *out);int  undo(Undo_Record *rec, Board *board);
```

- **Description:** pushMoveForUndo records the inverse of a move on the undo stack each time a move is executed. The undo button pops the top record and applies its inverse, restoring captured pieces, reverting castles, and replacing chains of Ants for Anteater captures.

Function 13: Quit

```
static void onQuitClicked   (GtkButton *b, gpointer ud);static void onQuitConfirmed
(GtkDialog *d, int id, gpointer ud);
```

- **Description:** Opens an "Are you sure?" confirmation dialog. On Yes, the GtkApplication is quit and a quit entry is appended to chess_game.log; on No, the dialog is dismissed and play continues.

Function 14: AI Move Selection

```
int aiSelectMoveAtDifficulty(Piece board[8][10], Color player,
AIDifficulty difficulty, Move *out);
```

- **Description:** Generates a single move for player at the given difficulty.
- **Logic:** AI_EASY picks a uniformly random legal move; AI_MEDIUM applies a one-ply heuristic that scores captures and king safety; AI_HARD runs alpha-beta minimax to depth 2 over the same heuristic. The legacy aiSelectMove() is preserved as a wrapper that defaults to AI_MEDIUM.
- **Outputs:** writes the chosen move to *out and returns 1 on success, 0 if the side has no legal moves.

Function 15: Piece Constructor

```
void pieceCtor(Piece *p, Icon *img, Color c, Rank type, Pos pos);
```

- **Description:** Constructs a Piece in place. Sets the image, color, and rank, looks up the matching PieceVTable for canMove and canCapture, sets the value field to the standard material value for that rank, copies pos, and initializes the moved flag to 0.

Function 16: Position Constructor

```
void posCtor(Pos *p, int row, int col);
```

- **Description:** Initializes a Pos with the given (row, col).

3.3 Detailed Description of Input and Output Formats

Input

All gameplay input is delivered through GTK 4 events:

- Left clicks on the board are routed through onClick to sendInput, which converts pixel coordinates into (row, col) using pixToIndex. The first click on a friendly piece selects it; the second click on a highlighted square executes the move.
- Buttons fire onUndoClicked (undo the last move), onQuitClicked (open the quit confirmation), and onPromotionClicked (apply the chosen promotion).
- The Game Setup dialog reads the user's choices for game mode, color, and CPU difficulty from three GtkComboBoxText widgets when the user clicks Start Game.

Internally, every move is normalized to a Move struct with start (row, col), end (row, col), and the capture / enPassant / castle flags.

Output

The program produces three streams of output:

- **Board redraw —** the GTK drawing area is invalidated whenever Board_State->hasUpdate is set, which causes drawBoard to repaint the squares, pieces, and legal-move highlights.
- **On-screen move log —** each completed move appends a line to the GtkTextView on the right of the window, e.g. "White: F2 -> F4", "Black: G7 -> G5 (en passant)", "Checkmate. White wins.". The text view auto-scrolls to the most recent entry.
- **chess_game.log file —** a tab-separated, human-readable log written to the working directory. A header is emitted by newLog at the start of each game; thereafter every action is appended via the corresponding log.c function. Sample contents:

```
Chess Game Log : Tue Apr 21 14:25:58 2026

#  Color  Action              Move
-----------------------------------------
1  White  Move                {A2, A4}
2  Black  CPU Move            {D7, D6}
3  White  Move                {B1, C3}
4  Black  Capture             {C8, F5}
End of game. Checkmate. White wins.
```

The save/load module (planned) will persist the entire Board, the current turn, and the contents of chess_game.log to a text file under save/, in a format that can be re-loaded to resume play exactly where it ended.

4.1 Partitioning of Tasks

TASK 1: GUI APPLICATION (gui/gui.c)

Hosts main() and the GtkApplication. Builds the main window with the board drawing area, move log, per-side timer labels, undo button, and quit button; presents the Game Setup dialog at launch and the Game Over dialog at end of game; coordinates the timer source and the CPU move callback.

TASK 2: GAME CONTROLLER (game_controller.c)

Owns sendInput, the single function that translates a click on the board into a complete game action. Decides whether to select a piece, execute an ordinary move, route to a special-move executor, or continue an Anteater chain capture, and tells the GUI when to open the promotion popover.

TASK 3: BOARD MANAGEMENT (board.c) AND BOARD STATE (board_state.c)

board.c owns the static board representation — initialization, lookup, mutation, captures, and promotion. board_state.c owns the per-turn dynamic state — the currently clicked piece, the legal-move list used for highlighting, last-move tracking for en passant, and update / game-over flags.

TASK 4: PIECE LOGIC (pieces.c)

Defines Piece, Pos, and PieceVTable. Implements every per-piece canMove and canCapture rule, the en passant and castling rule checks, and the king-in-check helpers (findKing, attackSquare, kingCheck). This is the authoritative chess rules module.

TASK 5: MOVE PROCESSING (move.c)

Defines Move and provides every executor used after a move has been validated: executeMove, executeCapture, executePawnCapture, executeAnteaterCapture, executeEnPassant, executeCastle, plus possibleMove which scans for the existence of any legal move (used for stalemate / checkmate detection).

TASK 6: RENDERER (gui/render.c) AND COLOR THEMING (gui/color_gui.c)

render.c uses Cairo to draw squares, pieces, and the green outlines for selected pieces and legal destination squares; loads the 14 piece textures from gui/resources/ at startup. color_gui.c centralizes board square colors and CSS class names so the visual theme can be tweaked in one place.

TASK 7: AI / COMPUTER PLAYER (ai.c)

Implements aiSelectMoveAtDifficulty across the three difficulties: AI_EASY picks a random legal move, AI_MEDIUM applies a one-ply capture-and-king-safety heuristic, and AI_HARD runs alpha-beta minimax to depth 2 over the same heuristic. The CPU's selected move is then executed through the same move.c pipeline as a human move.

TASK 8: LOGGING (log.c)

Maintains chess_game.log. Each entry is one line with move number, color, action category, and a coordinate-formatted move. The expanded log API distinguishes plain moves, captures, castles, en passant, promotions, check, checkmate, draws, and quits.

TASK 9: HISTORY MANAGER / UNDO (undo.c)

Stack-based history of executed moves, where each Undo_Record stores enough state to reverse exactly one move. Castling is handled by the second-piece fields (movedRook, rookMove); Anteater chain captures push one record per Ant consumed so undo unwinds the chain step-by-step.

TASK 10: SAVE / LOAD (save.c, planned)

Will write the entire Board, the current turn, and the contents of chess_game.log to a text file under save/ and reload it to resume play. The skeleton is in src/save.c; integration with the GUI is scheduled after the V1.0 freeze.

TASK 11: TESTING AND DEBUGGING

Two test drivers ship with the source tree: test_rulecheck.c exercises every piece's movement and capture rules across representative board positions, and test_boarddisplay.c verifies that the ASCII renderer produces the expected layout for a known starting position. Both run via make test.

TASK 12: DOCUMENTATION

Maintains README, INSTALL, the User Manual (Chess_UserManual.pdf), and this Software Specification (Chess_SoftwareSpec.pdf). Documents are updated each milestone to reflect the current code base.

4.2 Team Member Responsibilities

TEAM MEMBER	MAIN RESPONSIBILITIES	SHARED OR SECONDARY RESPONSIBILITIES
QUEENCY TAN	GTK 4 event wiring, board management (board.c), special moves and rule checks	game controller (game_controller.c), GUI design and layout, coding quality control
MIA NESS	logging (log.c), history management / undo (undo.c), check / checkmate / stalemate	game controller (game_controller.c), document quality control, coding quality control
AUSTIN NGUYEN	piece logic (pieces.c), special moves and rule checks, move processing (move.c)	game controller (game_controller.c), AI optimization, coding quality control
NAVEEN KHAN	input handling (input.c), AI / computer player (ai.c), documentation (title page, back matter, user manual revisions)	integration testing, coding quality control
AMAN MACWAN	graphical user interface (gui/gui.c), board bundle (board_bundle.c), per-side timer	game controller (game_controller.c), behavioral testing for pieces, coding quality control
GURVIN DHANJAL	renderer (gui/render.c), color theming (gui/color_gui.c), board state (board_state.c), build system	game controller (game_controller.c), Makefile maintenance, git control, coding quality control

WHILE SPECIFIC MODULES HAVE BEEN ASSIGNED TO INDIVIDUAL MEMBERS, ALL MEMBERS WILL MAKE CONTRIBUTIONS TO THE CREATION OF THE FINAL PRODUCT.

5. COPYRIGHT

© 2026 Team 21 — EECS 22L, University of California, Irvine. All rights reserved.

This document and the accompanying Anteater Chess software are produced as coursework for EECS 22L, Spring 2026. No part of this document or its associated source code may be reproduced, distributed, or transmitted in any form without the prior written permission of the authors, except for academic evaluation by course instructors and teaching assistants.

Anteater Chess is an original adaptation of the traditional chess game, developed exclusively for educational purposes at the University of California, Irvine. The Anteater piece and associated rules are defined by the EECS 22L course specification authored by Prof. Rainer Doemer.

6. REFERENCES

Doemer, R. "EECS 22L: Project 1 — The Game of Anteater Chess." University of California, Irvine, March 30, 2026.

GTK 4 — The GTK Project. https://www.gtk.org/

Cairo Graphics. https://www.cairographics.org/

GCC, the GNU Compiler Collection. Free Software Foundation. https://gcc.gnu.org/

Git — Distributed Version Control System. https://git-scm.com/

7. INDEX

A

AI / Computer Player (ai.c), 4, 14, 17
AIDifficulty, 10, 14, 17
Anteater Capture (Function 3), 13, 17
Anteater Chess, 1, 10, 17
anteaterHasAdjacentPawn, 13, 17

B

Bishop, 10
Board (struct), 11, 17
Board_Bundle, 12, 17
Board_State, 11, 17
Board Management (board.c), 4, 5, 17
Board State (board_state.c), 4, 5, 17

C

Cairo, 4, 17
Capture (Function 2), 13
Castle (Function 4), 13, 17
Check / Checkmate / Stalemate (Function 7), 14, 17
Color (enum), 10
Compile (make), 8, 9
CPU Difficulty, 10, 14, 17

E

End Game (Function 11), 14
En Passant (Function 5), 14, 17
Enumerated Types, 10

G

Game Controller (game_controller.c), 4, 5, 17
Game Setup Dialog, 7, 14
GCC Compiler, 8
Git (version control), 18
GTK 4, 4, 8
GUI Application (gui/gui.c), 4, 5, 17

H

Highlighting, 5, 7
Human vs CPU, 7, 14
Human vs Human, 7, 14

I

Input / Output Formats, 16
Installation, 8

K

King, 10
kingCheck, 6, 14
Knight, 10

L

Legal Move Enforcement, 5, 7
Linux, 8
Logging (log.c), 4, 6, 18

M

Make Utility, 8, 9
Makefile, 8, 9, 18
Move (Function 1), 13
Move struct, 11
Move Processing (move.c), 4, 5, 17

N

New Game (Function 10), 14

P

Pawn / Ant, 10
Per-Side Timer (Function 8), 14
Piece Constructor (Function 15), 15
Piece Logic (pieces.c), 4, 5, 17
Piece struct, 11
Piece_Type (enum), 10
PieceVTable, 11
Pos struct, 11
Position Constructor (Function 16), 15
Promotion (Function 6), 14, 17

Q

Queen, 10
Quit (Function 13), 15

R

Renderer (gui/render.c), 4, 6, 17
Rook, 10
Run Program (./bin/chess), 8

S

Save / Load (save.c), 4, 18
Software Architecture Overview, 3
Stalemate, 14
System Requirements, 8

T

Team Member Responsibilities, 19
Testing and Debugging, 18
Timer / Change Turn, 14
Turn (Function 9), 14

U

Undo (Function 12), 14
Undo_Record, 12
Uninstalling, 9